

6.15 Review questions

Understanding

1. What are the mandatory parts of a function header?
2. What is the difference between an argument and a parameter?
3. What does a function return if the return statement is omitted?
4. What is a docstring, and how is it read in REPL?
5. What happens when we try to change an `int` or `str` inside a function? Explain with reference to immutability.
6. What happens when we call `append()` on a list that has been passed in as an argument to a function?
7. What are keyword arguments, and what advantage do they provide?
8. Why should mutable objects be avoided as default values in function definitions?
9. What is the difference between `*args` and `**kwargs`?
10. What does LEGB mean in relation to scope in Python?
11. Why do `for` loops and `if` statements not create their own scope in Python, unlike e.g. Java?
12. What does `if __name__ == "__main__":` do and why is it useful?
13. What is `sys.argv`, and what does `sys.argv[0]` contain?
14. What is the difference between `*args` and `sys.argv` as techniques for receiving a variable number of values?
15. What does `Optional[str]` mean, and what is the equivalent modern syntax?
16. What is the difference between `list` and `list[int]` as a type hint?
17. What does `sys.exit(1)` do, and what does the number 1 mean?

Practical

18. Write a function `square(n)` that returns the square of a number. Add type hints and a docstring. Call the function with some different values and print the results.

19. Write a function `greet(name, greeting="Hello")` that prints a greeting. Test it both with and without the greeting argument.
20. Write a function `min_max(lst)` that returns both the smallest and largest value in a list. Use the functions `min()` and `max()`. Unpack the return values into two variables at the call site.
21. Write a function `count_above(lst, threshold)` that returns the number of elements in the list greater than `threshold`. Test with `[3, 7, 1, 9, 4]` and `threshold 5`.
22. Demonstrate that a list passed to a function can be modified directly: write a function `double_all(lst)` that doubles each element in place. Confirm that the original is changed after the call.
23. Write a function `sum_all(*numbers)` that sums an arbitrary number of numbers. Test it with single arguments and with unpacking of a list.
24. Write a script that takes a filename as a command-line argument and prints an error message if the argument is missing. Run it from the terminal with and without the argument.
25. Write a function `multiply` that takes a list of integers and an integer `factor` and returns a new list where each element is multiplied by the factor. Add type hints and a docstring. Test it with `[1, 2, 3, 4]` and `factor 3`.
26. Make a function that returns multiple values and unpack them on the call site.